

R for bioinformatics, data iteration & parallel computing

HUST Bioinformatics course series

Wei-Hua Chen (CC BY-NC 4.0)

HUST, China

2025-10-25

Table of contents I

- 1 section 1: TOC
- 2 section 2: iteration basics
- 3 section 3: iteration 进阶: the purrr package
- 4 section 4: 并行计算
- 5 section 5: 小结及作业 !

Section 1

section 1: TOC

前情提要

stringr, stringi and other string packages ...

① basics

- length
- uppercase, lowercase
- unite, separate
- string comparisons, sub string

② regular expression

本次提要

- for loop
- apply functions
- dplyr 的本质是 遍历
- map functions in purrr package
- 遍历 与 并行计算

Section 2

section 2: iteration basics

for loop , get data ready

```
library(tidyverse);
## create a tibble
df <- tibble( a = rnorm(100), b = rnorm(100), c = rnorm(100), d = rnorm(100) );

head(df, n = 3);
```

A tibble: 3 x 4

	a	b	c	d
	<dbl>	<dbl>	<dbl>	<dbl>
1	1.23	0.467	-0.792	0.466
2	1.09	0.342	-0.335	-0.417
3	0.0196	-0.832	-0.0978	-0.831

see for loop in action

```
## 计算 row means
res1 <- vector( "double", nrow(df) );
for( row_idx in 1:nrow( df ) ){
  res1[row_idx] <- mean( as.numeric( df[row_idx , ] ) );
}

res2 <- c();
for( row_idx in 1:nrow( df ) ){
  res2[ row_idx ] <- mean( as.numeric( df[row_idx , ] ) );
}

## 计算 column means
res2 <- vector( "double", ncol(df) );
for( col_idx in 1:ncol( df ) ){
  res2[col_idx] <- mean( df[[col_idx]] );
}
```


for loop 的替代

由于运行效率可能比较低，尽量使用 for loop 的替代

```
rowMeans( df );
```

```
[1] 0.3438405498 0.1706763722 -0.4354419036 -0.0539949265 -0.0066815127
[6] 0.2437468498 0.5712823211 0.4768942141 -0.1320603441 0.8228614320
[11] 0.8853051579 -0.0039498789 -0.0192338154 -0.5195945123 -0.2191435413
[16] 0.8526439913 0.6890005483 0.3421771004 0.4262910422 -0.1713292022
[21] -0.3334768178 0.7819947664 -0.0115007127 0.2190874435 0.0714641987
[26] -0.3326704200 0.5951078608 -0.5648590140 -0.3122565637 0.4487883892
[31] -0.1626726240 -0.4536084350 0.2323186575 0.5118517224 -0.1940578578
[36] -0.5156197196 0.3668824069 0.9001988852 0.0550649720 -0.8009099965
[41] -0.0993630379 0.1932736993 0.7305545512 -0.2714845786 0.3650307566
[46] -0.0953751336 0.5014601655 0.5474000387 0.3072531185 0.0449383736
[51] -0.3228424498 -0.1548847255 -0.6166026157 0.9443413491 -0.4489319676
[56] -0.1740919364 0.2969799138 0.2482450810 -0.5506476451 0.4123561653
[61] 0.3720032342 -0.2677127950 -0.1435864079 0.8688108661 0.5772649413
[66] -0.1558463014 0.0047658681 -0.3311529084 0.4026124532 0.2694567290
[71] -0.5040798429 0.5229599154 -0.2942381623 -0.2295365355 0.1189953949
[76] 0.0002211762 0.0741474436 -0.1166245682 0.2468395890 -0.2560494927
[81] 0.5744103007 0.0013155744 0.4152754345 0.7043353596 -0.1862382468
[86] 0.0000000000 0.0000000000 0.0000000000 0.0000000000 0.0000000000
```



apply 相关函数

Usage:

`apply(X, MARGIN, FUN, ...)`;

MARGIN : 1 = 行, 2 = 列; `c(1,2)` = 行 & 列

FUN : 函数, 可以是系统自带, 也可以自己写

```
df %>% apply(., 1, median ); ## 取行的 median
```

```
[1]  0.466675669  0.003244010 -0.464564985  0.189565259 -0.141237291
[6]  0.042399424  0.774170945  0.428644840 -0.084081840  0.687010589
[11]  0.574609402 -0.306042688 -0.260799307 -0.417953904 -0.220681495
[16]  1.069469193  0.683785833  0.555140209  0.456755045 -0.393885957
[21] -0.533370638  0.767531232 -0.102322724 -0.067498882  0.016334615
[26] -0.443929722  0.661924680 -0.659709568 -0.645184857  0.241583681
[31] -0.040929736 -0.391127089  0.472533318  0.339491499 -0.281652242
[36] -0.553075502  0.340670615  0.495386125  0.595110597 -0.622338965
[41] -0.006875946  0.531675031  0.987258760 -0.235243909 -0.314132919
[46] -0.166186634  0.615909583  0.567470734  0.025698943 -0.074193940
[51] -0.617005748 -0.294710621 -0.535558704  1.082251844 -0.548798193
[56] -0.245346308  0.296397701  0.208929036 -0.441794948  0.612956211
[61]  0.337832947 -0.163432891  0.036521951  0.808055446  1.032917386
```



apply 与自定义函数配合

```
df %>% apply( ., 2, function(x) {
  return( c( n = length(x), mean = mean(x), median = median(x) ) );
} ); ## 列的一些统计结果
```

	a	b	c	d
n	100.0000000	100.0000000	100.0000000	100.0000000
mean	0.2228010	0.1290224	0.06162319	0.09823749
median	0.1493552	0.0105668	-0.03146163	0.11563598

注意 行操作大部分可以被 dplyr 代替

tapply 的使用

以行为基础的操作，用法:

```
tapply(X, INDEX, FUN = NULL, ..., default = NA, simplify = TRUE)
```

用 **index** 将 **x** 分组后，用 **fun** 进行计算 -> 用 姓名 将 成绩 分组后，计算 平均值 用 汽缸数 将 油耗 分组后，计算 平均值

```
library(magrittr);
## 注意 pipe 操作符的使用
mtcars %>% tapply( mpg, cyl, mean ); ## 汽缸数 与 每加仑汽油行驶里程 的关系
```

```
      4          6          8
26.66364 19.74286 15.10000
```

tapply versus dplyr

然而，使用 dplyr 思路会更清晰

```
mtcars %>% group_by( cyl ) %>% summarise( mean = mean( mpg ) );
```

注意 tapply 和 dplyr 都是基于行的操作 !!

lapply 和 sapply

基于列的操作

输入:

- vector : 每次取一个 element
- data.frame, tibble, matrix : 每次取一列
- list : 每次取一个成员

lapply 和 sapply , cont.

输入是 tibble

```
df %>% lapply( mean );
```

\$a

[1] 0.222801

\$b

[1] 0.1290224

\$c

[1] 0.06162319

\$d

[1] 0.09823749

```
df %>% sapply( mean );
```

	a	b	c	d
	0.22280098	0.12902243	0.06162319	0.09823749



lapply 和 sapply , cont.

输入是 list , 使用自定义函数

```
list( a = 1:10, b = letters[1:5], c = LETTERS[1:8] ) %>%
  sapply( function(x) { length(x) } );
```

```
a  b  c
10 5  8
```

强调

- lapply 是针对列的操作
- 输入是 tibble, matrix, data.frame 时, 功能与 apply(x, 2, FUN) 类似 ...

Section 3

section 3: iteration 进阶: the purrr package

map , RStudio 提供的 lapply 替代



Figure 1: 来自 purrr package

- part of tidyverse

purrr 的基本函数

`map( FUN )` : 1. 遍历每列 (tibble) 或 slot (list), 2. 运行 FUN 函数, 3. 将计算结果返回至 list

对应: `lapply`

```
df %>% map( summary );
```

\$a

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-1.8985	-0.3856	0.1494	0.2228	1.0280	2.6795

\$b

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-2.00237	-0.61910	0.01057	0.12902	0.76386	3.14528

\$c

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-2.20451	-0.53904	-0.03146	0.06162	0.71064	2.03184

\$d

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-3.45301	-0.67968	0.11564	0.09824	0.88199	2.69241



对应 sapply 的 map_ 函数

- `map_lgl()` makes a logical vector.
- `map_int()` makes an integer vector.
- `map_dbl()` makes a double vector.
- `map_chr()` makes a character vector.

```
df %>% map_dbl( mean ); ## 注: 返回值只能是单个 double 值
```

```
      a      b      c      d
0.22280098 0.12902243 0.06162319 0.09823749
```

?? 以下代码运行结果会是什么???

```
df %>% map_dbl( summary );
df %>% sapply( summary );
```

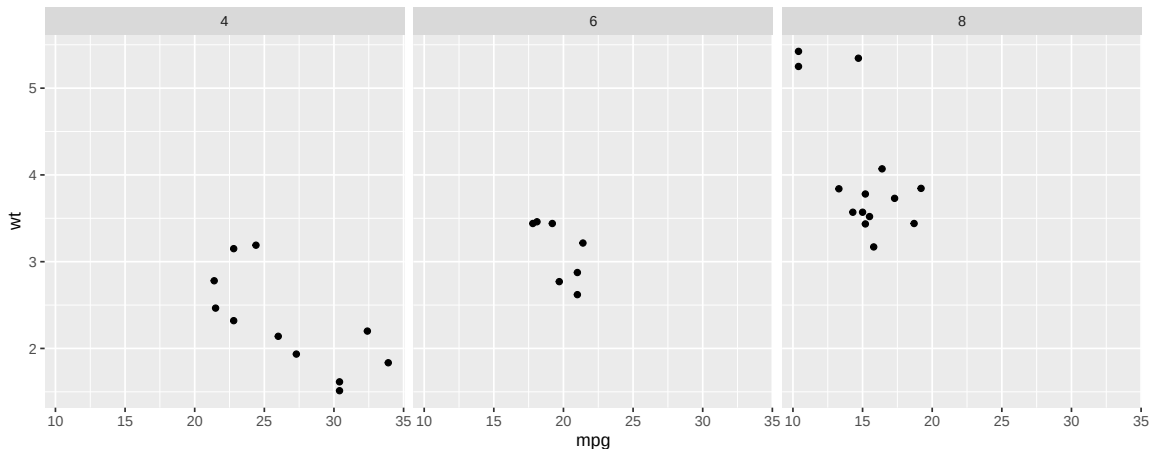
map 的高阶应用

为每一个汽缸分类计算: 燃油效率与吨位的关系

```
plt1 <-  
  mtcars %>%  
  ggplot( aes( mpg, wt ) ) +  
  geom_point( ) + facet_wrap( ~ cyl );
```

取得线性关联关系

```
plt1;
```



```
mtcars %>% split( .$cyl ) %>% map( ~ cor.test( .$wt, .$mpg ) ) %>% map_dbl( ~.$estimate );
```

4

6

8

命令详解

```
mtcars %>% split( .$cyl ) %>% map( ~ cor.test( .$wt, .$mpg ) ) %>% map_dbl( ~.$estimate );
```

- ❶ `split( .$cyl )`: 由 `purrr` 提供的函数, 将 `mtcars` 按 `cyl` 列分为三个 `tibble`, 返回值存入 `list`

注意: `.` 在 `pipe` 中代表从上游传递而来的数据; 在某些函数中, 比如 `cor.test()`, 必须指定输入数据, 可以用 `.` 代替。

请测试以下代码, 查看 `split` 与 `group_by` 的区别

```
mtcars %>% split( .$cyl );
mtcars %>% group_by( cyl );
```



命令详解, cont.

```
mtcars %>% split( .$cyl ) %>% map( ~ cor.test( .$wt, .$mpg ) ) %>% map_dbl( ~.$estimate );
```

map: 遍历上游传来的数据 (list), 对每个成分 (list 或 列) 运行函数: `~ cor.test( .$wt, .$mpg )`

注意

❶ 这里的 `cor.test` 应该有两种写法:

正规写法:

```
map( function(df) { cor.test( df$wt, df$mpg ) } )
```

简写:

```
map( ~ cor.test( .$wt, .$mpg ) )
```

❷ `~` 的用法: 用于取代 `function(df)`



命令详解, cont.

map 也可以进行数值提取操作: `map_dbl( ~.$estimate )`

上述命令同样有两种写法:

```
## 完整版  
map_dbl( function(eq) { eq$estimate} );  
## 简写版  
map_dbl( ~.$estimate )
```

more to read & exercise

- map: apply a function to **each element** of a **list**, return a **list**
- map2: apply a function to a **pair of elements**, return a **list**
- pmap: apply a function to groups of elements from a **list of lists or vectors**, return a **list**
- imap: ...
- more to read and exercise about iterations: <https://r4ds.had.co.nz/iteration.html>
 - filter
 - index
 - Modify
 - reshape
 - combine
 - reduce
- find more exercise at the end of the slides

reduce

```
dfs <- list(
  age = tibble(name = "John", age = 30),
  sex = tibble(name = c("John", "Mary"), sex = c("M", "F")),
  trt = tibble(name = "Mary", treatment = "A")
)

dfs %>% reduce(full_join)
```

```
# A tibble: 2 x 4
  name      age sex  treatment
  <chr> <dbl> <chr> <chr>
1 John      30 M      <NA>
2 Mary      NA F      A
```

reduce, cont.

```
vs <- list(  
  c(1, 3, 5, 6, 10),  
  c(1, 2, 3, 7, 8, 10),  
  c(1, 2, 3, 4, 8, 9, 10)  
)  
  
vs %>% reduce(intersect)
```

```
[1] 1 3 10
```

accumulate

```
( x <- sample(10) );
```

```
[1] 5 8 1 3 10 6 7 4 9 2
```

```
x %>% accumulate(`+`);
```

```
[1] 5 13 14 17 27 33 40 44 53 55
```

Section 4

section 4: 并行计算

并行计算介绍

并行计算一般需要 3 个步骤:

- ① 分解并发放任务
- ② 分别计算
- ③ 回收结果并保存

相关的包

`parallel` 包: 检测 CPU 数量;

`doParallel` 包: 将全部或部分 分配给任务

`foreach` 包: 提供 `%do%` 和 `%dopar%` 操作符, 以提交任务, 进行顺序或并行计算

辅助包:

`iterators` 包: 将 `data.frame`, `tibble`, `matrix` 分割为行/列 用于提交并行任务。

注意 任务完成后, 要回收分配的 CPU core。

首先安装相关包 (一次完成)。

```
install.packages("doParallel");  
install.packages("foreach"); ## 会自动安装 iterators
```



简单示例

```
library(doParallel); ##  
library(foreach);  
library(iterators);  
  
## 检测有多少个 CPU --  
( cpus <- parallel::detectCores() );
```

```
[1] 10
```

```
registerDoParallel( cpus - 1 );
```

简单示例, cont.

```
## Basic loop
data(iris)
x <- iris[which(iris[,5] != "setosa"), c(1,5)]
trials <- 10000

start <- proc.time()
outdf <- data.frame()
r <- for (i in 1:trials){
  ind <- sample(100, 100, replace=TRUE)
  result1 <- glm(x[ind,2]~x[ind,1], family=binomial(logit))
  outdf <- rbind(outdf,coefficients(result1))
}

base_loop <- proc.time()-start
```

简单示例, cont.

%do% loop - foreach notation, but not parallel

```
start <- proc.time()
r <- foreach(icount(trials), .combine=rbind) %do% {
  ind <- sample(100, 100, replace=TRUE)
  result1 <- glm(x[ind,2]~x[ind,1], family=binomial(logit))
  coefficients(result1)
}
do_loop <- proc.time()-start
```

简单示例, cont.

%dopar% adds parallelization

```
start <- proc.time()
r <- foreach(icount(trials), .combine=rbind) %dopar% {
  ind <- sample(100, 100, replace=TRUE)
  result1 <- glm(x[ind,2]~x[ind,1], family=binomial(logit))
  coefficients(result1)
}
dopar_loop <- proc.time()-start
```

简单示例, cont.

结果比较:

```
print(rbind(base_loop,do_loop,dopar_loop)[,1:3])
```

	user.self	sys.self	elapsed
base_loop	4.961	0.275	5.237
do_loop	4.505	0.136	4.641
dopar_loop	0.292	0.057	1.323

命令详解

`.combine = 'c'` 参数的可能值:

- 'c': 将返回值合并为 vector ; 当返回值是单个数字或字符串的时候使用
- 'cbind': 将返回值按列合并
- 'rbind': 将返回值按行合并
- 默认情况下返回 list

数据分发练习

将下面的计算转为并行计算

```
mtcars %>% split( .$cyl ) %>% map( ~ cor.test( .$wt, .$mpg ) ) %>% map_dbl( ~.$estimate );
```

```
## make a cluster --
```

```
cl2 <- makeCluster( cpus - 1 );
```

```
registerDoParallel(cl2);
```

```
## 分配任务 ...
```

```
res2 <- foreach( df = iter( mtcars %>% split( .$cyl ) ), .combine = 'rbind' ) %dopar% {
  cor.res <- cor.test( df$wt, df$mpg );
  return ( c( cor = cor.res$estimate, p = cor.res$p.value ) ); ## 注意这里的返回值是
}
```

```
res3 <- foreach( df = iter( mtcars %>% split( .$cyl ) ), .packages = c("ggplot2") ) %dopar% {
  p <- ggplot(df, aes( x = wt, y = mpg )) + geom_point();
  return ( p ); ## 注意这里的返回值是
}
```

```
## 注意在最后关闭创建的 cluster
```

```
stopCluster( cl2 );
```

练习详解

- ① `df = iter( mtcars %>% split( .$cyl ) )`: `mtcars` 按汽缸数分割为 3 个 list, 依次赋予 `df`;
- ② `cor.res <- cor.test( df$wt, df$mpg )`: 计算每个 `df` 中 `wt` 与 `mpg` 的关联, 将结果保存在 `cor.res` 变量中;
- ③ `.combine = 'rbind'`: 由于返回值是 `vector`, 用此命令按行合并;

foreach 的其它参数

`.packages=NULL` : 将需要的包传递给任务。如果每个任务需要提前装入某些包, 可以此方法。
比如:

```
.packages=c("tidyverse")
```

嵌套 (nested) foreach

有些情况下需要用到嵌套循环，使用以下语法：

```
foreach( ... ) %:% {  
  foreach( ... ) %dopar% {  
  
  }  
}
```

即：外层的循环部分用%:% 操作符

其它并行计算函数

parallel 包本身也提供了 lapply 等函数的并行计算版本, 包括:

- parLapply
- parSapply
- parRapply
- parCapply

parLapply 举例

任务: 计算 2 的 N 次方:

```
cl<-makeCluster(3);  
parLapply(cl,  
          2:4,  
          function(exponent)  
            2^exponent);  
stopCluster(cl);
```

其它的函数这里就不一一介绍了

Section 5

section 5: 小结及作业 !

本次小结

iterations 与 并行计算

- for loop
- apply functions
- dplyr 的本质是 遍历
- map functions in purrr package
- 遍历 与 并行计算

相关包

- purrr
- parallel
- foreach
- iterators

下次预告

data visualizations

- basic plot functions
- basic ggplot2
- special letters
- equations

作业

- Exercises and homework 目录下 talk08-homework.Rmd 文件;
- 完成时间: 见钉群的要求